Learn. Build. Solve. Repeat.

v1.0 ★ Edition

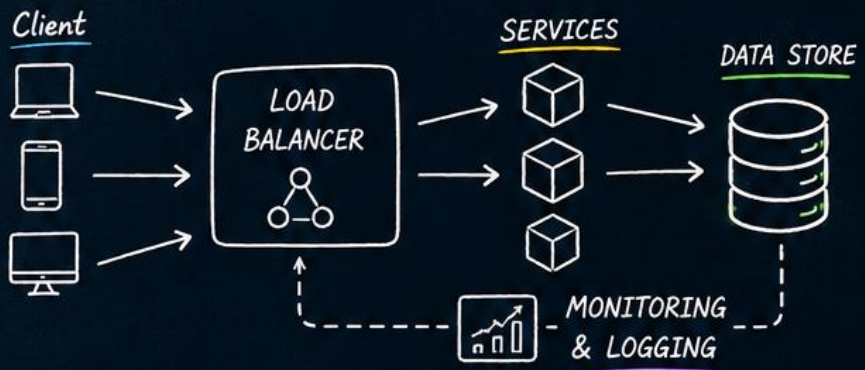
Code + Design = Impact

BACKEND + INTERVIEW QUESTIONS

HANDWRITTEN HANDBOOK

CONCEPTS • DIAGRAMS • STRATEGIES • BEST PRACTICES

SYSTEM DESIGN | DATABASE | SCALABILITY | SECURITY | PERFORMANCE



- ☑ Scalable
- ☑ Reliable
- ☑ Available
- ☑ Maintainable
- ☑ Secure

"Good code works. Great design scales."

CRACK INTERVIEWS
BUILD SYSTEMS
CREATE IMPACT

20 PAGES
Ultimate Guide for Developers

Understand | Design | Implement | Scale | Succeed

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

COMPLETE HANDWRITTEN NOTES

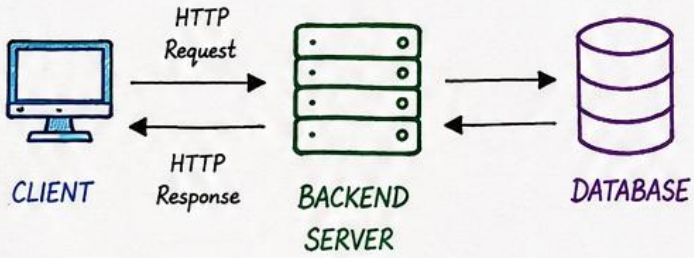
1. BACKEND FUNDAMENTALS

Backend is the server-side part of an application that handles business logic, database operations, security and API management.

What does Backend do?

- Processes client requests.
- Applies business logic.
- Stores and retrieves data from database.
- Handles authentication & authorization.
- Ensures security and data validation.
- Returns appropriate responses to client.

BASIC BACKEND ARCHITECTURE



KEY CHARACTERISTICS

- → Server-Side Logic : All core logic runs on the server.
- → Database Interaction : Communicates with DB to store/retrieve data.
- → API Handling : Exposes APIs for frontend or other services.
- → Security : Protects data and controls access.
- → Scalability : Designed to handle large number of users.
- → Reliability : Ensures high availability and performance.

COMMON BACKEND TECH STACK

- Languages : Java, Python, JavaScript (Node.js), Go, C#, PHP
- Frameworks : Spring Boot, Django, Express.js, .NET, FastAPI
- Databases : MySQL, PostgreSQL, MongoDB, Redis
- Others : Docker, Nginx, Kafka, RabbitMQ

TYPES OF BACKEND

- Monolithic Architecture : Entire application in a single codebase.



- Microservices Architecture : Application is divided into small, independent services.



- Serverless Architecture : Focus on writing code. Server management is handled by cloud provider.



HOW BACKEND WORKS ?

1. Client sends a request to the server (via API / URL).
2. Server receives the request.
3. Server validates the request.
4. Applies business logic.
5. Interacts with database / external services if needed.
6. Processes the data.
7. Sends response back to the client.

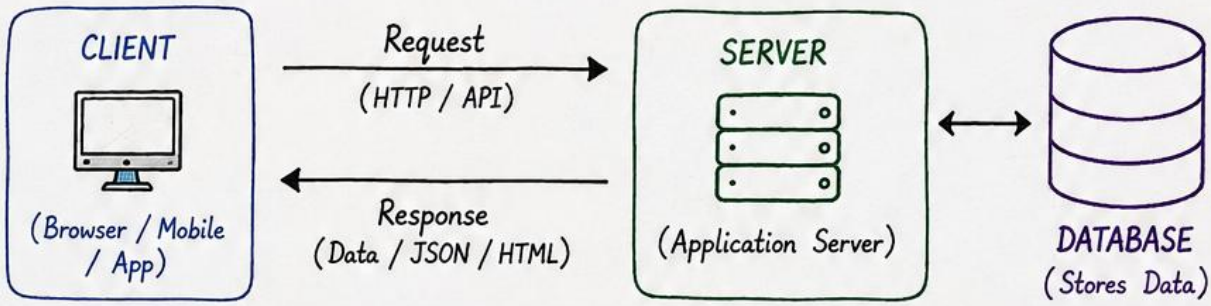
A strong backend is the backbone of any powerful application!

REMEMBER : Great UI attracts users, but Great Backend keeps them coming back! ★

BACKEND + INTERVIEW QUESTIONS

2. CLIENT – SERVER ARCHITECTURE

Client-Server architecture is a distributed application structure where clients request services and servers provide them.



HOW IT WORKS ?

1. Client sends a request to the server.
2. Server processes the request.
3. Server interacts with database (if needed).
4. Server sends the response back to the client.

BENEFITS

- Separation of concerns.
- Scalability.
- Multiple clients can connect simultaneously.
- Centralized data management.

CLIENT vs SERVER RESPONSIBILITIES

CLIENT	SERVER
User Interface	Business Logic
Sends Requests	Processes Requests
Displays Response	Returns Response
Validation (Basic)	Validation (Advanced)
No Direct DB Access	Database Access
Example: Browser / App	Example: Node.js / Java / Python Server

EXAMPLES

- Web Browser (Client) ⟺ Web Server
  (Chrome, Firefox) (Nginx, Apache)
- Mobile App (Client) ⟷ API Server
  (Android, iOS) (Node.js, Django)
- Postman (Client) ⟷ REST API
  (API Testing Tool) (Express.js, Spring Boot)

COMMON PROTOCOLS

- HTTP / HTTPS : For web communication.
- TCP / IP : For reliable data transfer.
- WebSocket : For real-time, two-way communication.

KEY POINTS TO REMEMBER

- ★ Client never communicates directly with database.
- ★ Server acts as an intermediary.
- ★ Entire communication happens over network.
- ★ APIs are the bridge between client and server.

INTERVIEW TIP

Always explain with a simple diagram, real-life example, and key benefits.
Keep it short, clear and to-the-point!

BACKEND + INTERVIEW QUESTIONS

3. HTTP – HYPERTEXT TRANSFER PROTOCOL

HTTP is the foundation of data communication on the Web. It defines how messages are formatted and transmitted between clients and servers.

HTTP METHODS

METHOD	DESCRIPTION
GET	Request data from server
POST	Send data to server to create
PUT	Update entire resource
PATCH	Update part of resource
DELETE	Delete a resource
HEAD	Same as GET but headers only
OPTIONS	Get supported methods

HTTP STATUS CODES

1xx Informational
- 100 Continue

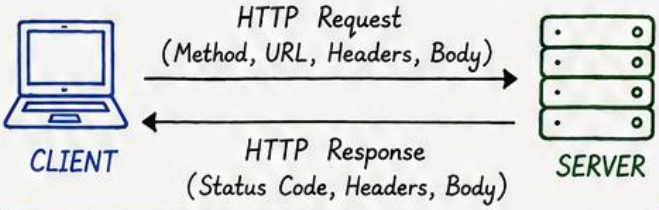
2xx Success
- 200 OK
- 201 Created

3xx Redirection
- 301 Moved Permanently
- 304 Not Modified

4xx Client Error
- 400 Bad Request
- 404 Not Found

5xx Server Error
- 500 Internal Server Error
- 502 Bad Gateway

HTTP REQUEST – RESPONSE FLOW



COMPONENTS OF HTTP REQUEST

- Request Line : Method + URL + HTTP Version
- Headers : Metadata about the request
- Body : Data sent to server (optional)
- Example (POST Request):

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer <token>
{
  "name": "Nikhil",
  "email": "nikhil@example.com"
}
```

COMPONENTS OF HTTP RESPONSE

- Status Line : HTTP Version + Status Code
- Headers : Metadata about response
- Body : Actual data returned
- Example (Response):

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 45
{
  "id": 101,
  "name": "Nikhil",
  "email": "nikhil@example.com"
}
```

KEY TAKEAWAYS

- ✓ HTTP is stateless.
- ✓ Every request is independent.
- ✓ Use correct HTTP methods.
- ✓ Understand status codes.

★

INTERVIEW TIP

Interviewers love to test your clarity on REST API basics, HTTP methods and status codes.

Be clear, be confident! ☺

BACKEND + INTERVIEW QUESTIONS

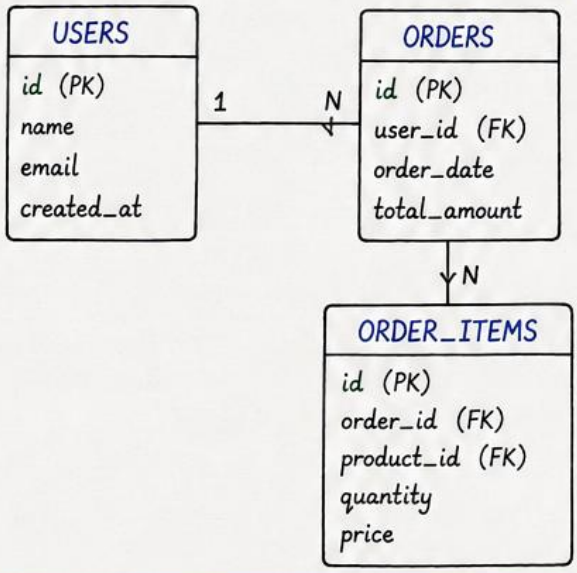
4. DATABASES & SQL FUNDAMENTALS

A database is an organized collection of data that can be easily accessed, managed and updated.

TYPES OF DATABASES

- Relational Database (RDBMS) : MySQL, PostgreSQL, SQLite, SQL Server
- NoSQL Database : MongoDB, Cassandra, Redis, DynamoDB
- In-Memory Database : Redis, Memcached
- Graph Database : Neo4j

RELATIONAL DATABASE EXAMPLE



SQL COMMANDS CHEAT SHEET

- SELECT : Retrieve data from table
- INSERT : Insert new data into table
- UPDATE : Update existing data in table
- DELETE : Delete data from table
- WHERE : Filter records
- ORDER BY : Sort records
- GROUP BY : Group records
- HAVING : Filter after grouping
- JOIN : Combine rows from two or more tables

KEY SQL CLAUSES

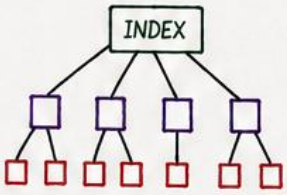
- WHERE → Filter rows before grouping
- GROUP BY → Group rows with same values
- HAVING → Filter groups after grouping
- ORDER BY → Sort result set

SQL JOIN TYPES

JOIN TYPE	DESCRIPTION
INNER JOIN	Returns rows that have matching values in both tables.
LEFT JOIN	Returns all rows from left table and matched rows from right table.
RIGHT JOIN	Returns all rows from right table and matched rows from left table.
FULL JOIN	Returns all rows when there is a match in either left or right table.

INDEXING

- Index is a data structure that improves the speed of data retrieval operations.
- Helps in faster SEARCH and reduces query execution time.
- Types :
  - Primary Index
  - Unique Index
  - Composite Index



KEY TAKEAWAYS

- ✓ Choose the right database for your use case.
- ✓ Normalize data to reduce redundancy.
- ✓ Use indexes wisely.
- ✓ Write optimized queries.
- ✓ Always backup your data.

INTERVIEW TIP

Understand how data is stored, retrieved and managed. Strong SQL skills are a must for every backend developer!

BACKEND + INTERVIEW QUESTIONS

5. API DESIGN – REST PRINCIPLES

REST (Representational State Transfer) is an architectural style for designing networked applications.

REST PRINCIPLES

- **Client–Server Separation**
  UI and data storage are separate.
- **Stateless**
  Each request from client must contain all the information.
- **Cacheable**
  Responses should define themselves as cacheable or not.
- **Uniform Interface**
  Standard methods, status codes and data formats.
- **Layered System**
  Client doesn't know if it is connected directly to the end server or an intermediate server.
- **Code on Demand (optional)**
  Server can extend client functionality by sending executable code.

HTTP METHODS – CRUD MAPPING

METHOD	ACTION	DESCRIPTION	EXAMPLE
GET	READ	Retrieve a resource	GET /users
POST	CREATE	Create a new resource	POST /users
PUT	UPDATE	Update entire resource	PUT /users/1
PATCH	UPDATE	Update part of resource	PATCH /users/1
DELETE	DELETE	Delete a resource	DELETE /users/1

EXAMPLE – USER RESOURCE ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/api/users	Get all users
GET	/api/users/{id}	Get user by id
POST	/api/users	Create new user
PUT	/api/users/{id}	Update full user
PATCH	/api/users/{id}	Update partial user
DELETE	/api/users/{id}	Delete user

COMMON HTTP STATUS CODES

1xx	Informational	Request received, continuing process
2xx	Success	Request successfully received, understood and accepted
3xx	Redirection	Further action needs to be taken
4xx	Client Error	Issue with client request
5xx	Server Error	Issue with server

TYPICAL API RESPONSE STRUCTURE

```
{
  "status": "success",                          ← Result of request
  "message": "User retrieved successfully",     ↔ Human readable message
  "data": {
    "id": 1,
    "name": "Nikhil",                           ← Actual response payload
    "email": "nikhil@example.com"
  }
}
```

API DESIGN BEST PRACTICES

- ✓ Use nouns, not verbs in URLs.
- ✓ Use plural for resource names.
- ✓ Keep URLs short and meaningful.
- ✓ Use proper HTTP methods.
- ✓ Always return proper status codes.
- ✓ Implement pagination for large data.
- ✓ Version your API (e.g., /api/v1/users).



INTERVIEW TIP

Explain REST principles with real examples. Interviewers love when you connect theory with practical use cases.





KEY TAKEAWAY: Good API design makes your system scalable, maintainable and easy to use. Follow REST principles and keep it *simple!*

BACKEND + INTERVIEW QUESTIONS

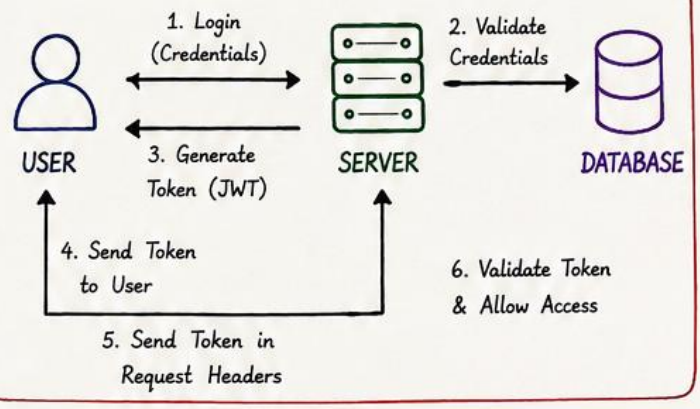
6. AUTHENTICATION & AUTHORIZATION

Authentication verifies *who you are.*
Authorization decides *what you can do.*

AUTHENTICATION vs AUTHORIZATION

AUTHENTICATION	AUTHORIZATION
• Verifies identity of user.	• Verifies permissions of user.
• Answers: "Who are you?"	• Answers: "What can you do?"
• Happens first.	• Happens after authentication.
• Example: Login with email & password.	• Example: Accessing admin panel.

AUTH FLOW (HIGH LEVEL)



WAYS TO AUTHENTICATE

- Username & Password : Traditional method.
- JWT (JSON Web Token) : Stateless, token based.
- OAuth 2.0 : Social login (Google, GitHub).
- API Keys : For third-party services.
- Multi-Factor Auth (MFA) : Extra layer of security.

JWT (JSON WEB TOKEN) STRUCTURE

HEADER •••• PAYLOAD •••• SIGNATURE

- **HEADER**: Contains token type & signing algorithm.
- **PAYLOAD**: Contains claims (user info, roles, permissions, etc.)
- **SIGNATURE**: Used to verify that token is valid & not tampered.

Final JWT = Base64Url(HEADER).Base64Url(PAYLOAD).SIGNATURE

JWT EXAMPLE

HEADER (Base64Url)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD (Base64Url)

```
{
  "id": 101,
  "name": "Nikhil",
  "role": "admin",
  "exp": 1717000000
}
```

SIGNATURE

```
dJ1m9...Xyz123
```

SESSIONS vs JWT

FEATURE	SESSIONS	JWT
Storage	Stored on server	Stored on client
State	Stateful	Stateless
Scalability	Harder to scale	Easy to scale
Performance	More server load	Less server load
Revocation	Easy (delete session)	Harder (use blacklist)
Use Case	Web apps	APIs, Mobile apps, Microservices

BEST PRACTICES

- ✓ Always use HTTPS.
- ✓ Hash passwords using strong algorithms (bcrypt).
- ✓ Use strong & unique secrets for JWT.
- ✓ Set token expiration time.
- ✓ Implement refresh tokens for better security.
- ✓ Follow least privilege principle.

INTERVIEW TIP

Interviewers love asking about JWT, sessions, OAuth, and the difference between authentication and authorization. Be clear with examples!

KEY TAKEAWAY: Authenticate the user to know who they are, Authorize the user to decide what they can do. ★

BACKEND + INTERVIEW QUESTIONS

7. DATABASES & SQL ESSENTIALS

Databases store, organize and manage data efficiently.

TYPES OF DATABASES

- Relational DB (RDBMS) : MySQL, PostgreSQL, SQL Server, Oracle
- NoSQL DB : MongoDB, Cassandra, Redis
- In-Memory DB : Redis, Memcached
- Columnar DB : ClickHouse
- Graph DB : Neo4j

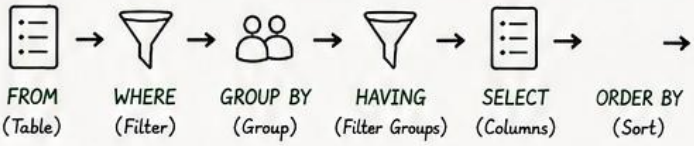
RDBMS vs NoSQL

RDBMS	NoSQL
• Table based (Rows & Columns)	• Document / Key-Value / Column / Graph based
• Schema is fixed	• Schema is flexible
• ACID compliant	• BASE (Basically Available Soft state, Eventually consistent)
• SQL language	• NoSQL queries / APIs
• Example: MySQL	• Example: MongoDB

SQL – BASIC COMMANDS

COMMAND	DESCRIPTION
SELECT	Retrieve data from table
INSERT	Insert new data into table
UPDATE	Update existing data
DELETE	Delete data from table
WHERE	Filter records
ORDER BY	Sort records
GROUP BY	Group records
HAVING	Filter groups
LIMIT	Limit number of results

SQL QUERY FLOW



Example Query:

```
SELECT department, COUNT(*) as total_emp
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 5
ORDER BY total_emp DESC
LIMIT 10;
```

Executed in the order shown in the flow.

NORMALIZATION (RDBMS)

- Process of organizing data to reduce redundancy and improve data integrity.
- Normal Forms:
  - 1NF : Atomic values (no repeating groups)
  - 2NF : No partial dependency
  - 3NF : No transitive dependency
  - BCNF : Stronger version of 3NF



PRIMARY KEY vs FOREIGN KEY

PRIMARY KEY

- Uniquely identifies each row
- Cannot be NULL
- One primary key per table



FOREIGN KEY

- Refers to primary key of another table
- Can have duplicate values
- Used to maintain relationships

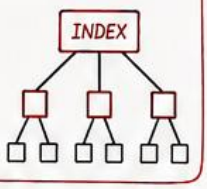


JOINS IN SQL

INNER JOIN		Returns matching rows from both tables.
LEFT JOIN		Returns all rows from left table and matching from right.
RIGHT JOIN		Returns all rows from right table and matching from left.
FULL JOIN		Returns all rows when there is a match in either left or right.

INDEXES

- Index is a data structure that improves the speed of data retrieval.
- Types:
  - Primary Index : Automatically created for PK
  - Unique Index : Ensures all values are unique
  - Composite Index : Index on multiple columns



KEY TAKEAWAYS

- ✓ Choose the right database for your use case.
- ✓ Use SQL to interact with relational data.
- ✓ Indexes and normalization improve performance.
- ✓ Understand joins and keys – very important!

INTERVIEW TIP



Practice writing SQL queries daily. Solve real problems on platforms like LeetCode, HackerRank and SQLZoo.



BACKEND + INTERVIEW QUESTIONS

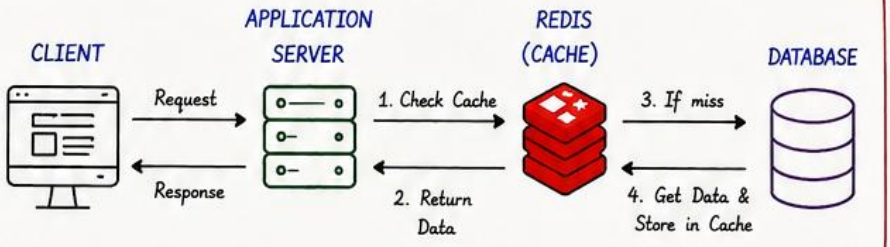
8. CACHING & REDIS BASICS

Caching stores frequently accessed data in fast storage (memory) so that we can reduce database load and improve response time.

WHY CACHE?

- Improves performance (faster responses)
- Reduces database load
- Handles more traffic
- Saves cost
- Improves user experience

CACHE ARCHIECTURE



WHAT IS REDIS?

- Redis = REmote DIctionary Server
- In-memory data structure store
- Very fast (works in memory)
- Supports strings, lists, sets, hashes, sorted sets, bitmaps, hyperloglog, etc.
- Used for caching, sessions, leaderboards, rate limiting, queues, pub/sub, etc.

REDIS DATA TYPES (COMMON)

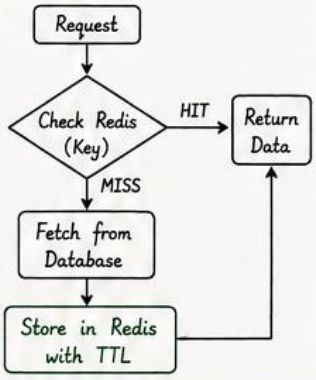
DATA TYPE	DESCRIPTION	EXAMPLE
String	Key → Value (text or number)	SET name "Nikhil"
Hash	Key → Field → Value	HSET user:1 name Nikhil
List	Ordered list of strings	LPUSH tasks "task1"
Set	Unordered collection of unique strings	SADD tags nodejs
Sorted Set	Set with scores (sorted)	ZADD leaderboard 100 nikhil

CACHE STRATEGIES

- **Cache-Aside (Lazy Loading)**
  Application checks cache first. If miss, fetches from DB and stores in cache.
- **Write-Through**
  Write to cache and DB at the same time.
- **Write-Behind (Write-Back)**
  Write to cache first, then to DB (asynchronously).
- **Cache Invalidation**
  Remove or update cache when data changes.

CACHE-ASIDE FLOW (EXAMPLE)

1. App receives request for data (e.g., user:1)
2. Check Redis:
   - If HIT → return data
   - If MISS → go to DB
3. Fetch data from DB
4. Store data in Redis with TTL
5. Return data to client



REDIS BASIC COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
SET key value	Set key to value	SET name "Nikhil"
GET key	Get value by key	GET name
DEL key	Delete key	DEL name
EXPIRE key sec	Set TTL (in seconds)	EXPIRE name 3600
KEYS pattern	Find keys matching pattern	KEYS user:*

CODE EXAMPLE (NODE.JS + REDIS)

```
const redis = require('redis');
const client = redis.createClient();
(async () => {
  await client.connect();
  await client.set('name', 'Nikhil', { EX: 3600 }); // set with TTL
  const value = await client.get('name');          // get value
  console.log('Name:', value);
  await client.disconnect();
})();
```

KEY TAKEAWAYS

- ✓ Cache reduces DB load and improves performance.
- ✓ Redis is an in-memory data structure store.
- ✓ Use the right strategy for your use case.
- ✓ Always set TTL to avoid stale data.
- ✓ Invalidate cache when data is updated.

INTERVIEW TIP

Interviewers love asking about cache strategies, Redis data types and when to use them. Be ready with real-world examples!

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

9. INDEXING & QUERY OPTIMIZATION

Indexing helps the database find data faster without scanning the entire table.
It improves read performance at the cost of extra storage and slower writes.

WHAT IS AN INDEX?

- An index is a data structure that improves the speed of data retrieval operations.
- Works like the index in a book.
- DB uses the index to locate rows quickly instead of scanning the whole table.



HOW INDEX WORKS (EXAMPLE)

INDEX (on name)

NAME	POINTER
Aman	0x01
John	0x02
Nikhil	0x03
Zara	0x04

TABLE (Users)

ID	NAME	EMAIL
1	Aman	aman@example.com
2	John	john@example.com
3	Nikhil	nikhil@example.com
4	Zara	zara@example.com
...		

DB uses index to jump directly to the row instead of scanning one by one.

TYPES OF INDEXES

INDEX TYPE	DESCRIPTION
Primary Index	Created automatically on PRIMARY KEY. Unique and clustered.
Unique Index	Ensures all values are unique.
Composite Index	Index on multiple columns.
Clustered Index	Sorts and stores table data based on index order.
Non-Clustered Index	Separate structure from table data.
Full-Text Index	Used for full-text search.

WHEN TO USE INDEX?

- ✓ Columns used in WHERE clause
- ✓ Columns used in JOIN conditions
- ✓ Columns used in ORDER BY / GROUP BY
- ✓ Columns that are frequently searched
- ✓ Columns with high cardinality (many unique values)



WHEN NOT TO USE INDEX?

- ✗ Small tables (overhead > benefit)
- ✗ Columns with very few unique values (e.g., gender)
- ✗ Frequent INSERT / UPDATE / DELETE operations
- ✗ Columns not used in search or filtering

EXPLAIN QUERY PLAN

Use EXPLAIN to understand how DB executes a query.

```
EXPLAIN SELECT *
FROM users
WHERE name = 'Nikhil';
```

Returns Execution Plan

ID	SELECT_TYPE	TABLE	TYPE	ROWS	EXTRA
1	SIMPLE	users	ref	1	Using index

- type = ref → Index is used (good)
- type = ALL → Full table scan (bad)
- rows → Estimated rows to be scanned
- extra → Additional information

QUERY OPTIMIZATION TIPS

- Select only required columns (avoid SELECT *)
- Use proper indexes
- Avoid functions on indexed columns in WHERE (e.g., WHERE YEAR(created_at) = 2024 ✗)
- Use WHERE before GROUP BY, HAVING
- Avoid OR in WHERE (use UNION if needed)
- Paginate large results (LIMIT & OFFSET)
- Keep statistics updated (ANALYZE TABLE)



SAMPLE SCENARIO

Query : SELECT * FROM orders WHERE user_id = 101 ORDER BY created_at DESC, LIMIT 10;

Optimization :

1. Create index on (user_id, created_at)
2. DB uses index to filter + sort
3. Returns first 10 rows fast ✓



KEY TAKEAWAYS



- ✓ Indexes improve read performance.
- ✓ Too many indexes slow down writes.
- ✓ Use indexes wisely based on queries.
- ✓ Always analyze and optimize slow queries.
- ✓ Understand the trade-off before adding indexes.



INTERVIEW TIP Be ready to explain different index types, when to use them, how they impact performance and how you would optimize a slow query.



BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

10. SYSTEM DESIGN BASICS

System design is about designing scalable, reliable and efficient systems that can handle large scale traffic.

KEY QUALITIES OF GOOD SYSTEM

- **Scalability** – Handle growth
- **Reliability** – High availability & fault tolerant
- **Performance** – Low latency, high throughput
- **Availability** – System is always up
- **Maintainability** – Easy to update & debug
- **Security** – Protect data & privacy
- **Cost Efficiency** – Optimize resources & cost

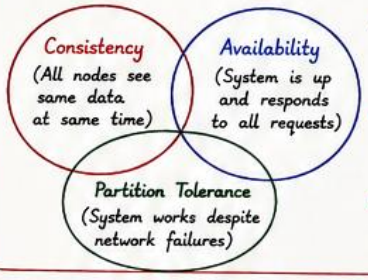
COMMON SYSTEM COMPONENTS

Client	Load Balancer	API Gateway	Application Server
Cache (Redis)	Database (MySQL)	Message Queue (Kafka)	Search Engine (Elasticsearch)
CDN	Storage (S3)	Monitoring (Prometheus)	Logging (ELK)

TRADE-OFFS (EXAMPLES)

- Consistency vs Availability (CAP Theorem)
- SQL (Consistency) vs NoSQL (Scalability)
- Caching (Faster) vs Fresh Data (Real-time)
- Cost vs Performance
- Simplicity vs Flexibility

CAP THEOREM (IN SIMPLE WORDS)



- You can achieve only two at a time.
- Most systems choose AP or CP.
- Very few can achieve all three simultaneously.

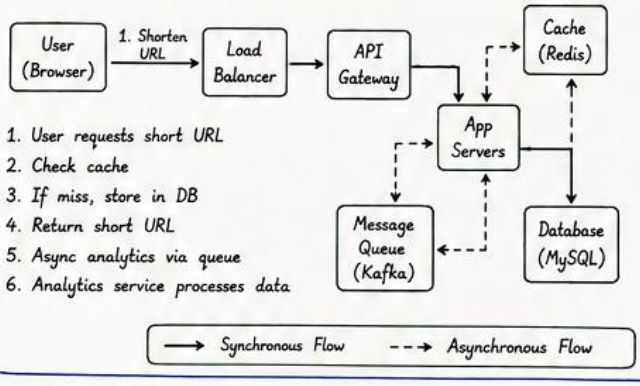
COMMON BOTTLENECKS

- Slow Database Queries
- High Latency in External APIs
- Insufficient Cache
- Large File Uploads / Downloads
- Too many Synchronous Calls

SYSTEM DESIGN PROCESS

1. **Understand Requirements**
   Clarify functional & non-functional requirements.
2. **High Level Design**
   Identify major components & their interactions.
3. **Detailed Design**
   Design components, APIs, databases, data flow.
4. **Data Design**
   Choose databases, schemas, indexing, caching.
5. **Scalability & Performance**
   Identify bottlenecks & scale the system.
6. **Reliability & Fault Tolerance**
   Replication, backup, monitoring, alerts.
7. **Cost Estimation & Trade-offs**
   Balance cost with performance and reliability.
8. **Summarize & Communicate**
   Present design and justify trade-offs.

EXAMPLE: DESIGN URL SHORTENER (HIGH LEVEL)



1. User requests short URL
2. Check cache
3. If miss, store in DB
4. Return short URL
5. Async analytics via queue
6. Analytics service processes data

SCALING TYPES

Vertical Scaling (Scale Up)		Add more power (CPU, RAM) to a single server.
Horizontal Scaling (Scale Out)		Add more servers behind load balancer.
Auto Scaling		Automatically add/remove servers based on traffic.

BEST PRACTICES

- ✓ Design for failure.
- ✓ Monitor everything.
- ✓ Automate scaling.
- ✓ Keep components loosely coupled.
- ✓ Review and improve continuously.

INTERVIEW TIP Always start with requirements, make assumptions, design step by step and communicate clearly. There is no 100% perfect design!

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

11. API DESIGN BEST PRACTICES

Good API design makes your system easy to use, scalable and developer friendly.

PRINCIPLES OF GOOD API DESIGN

- **Consistent** – Follow common conventions.
- **Simple** – Easy to understand & integrate.
- **Resource Oriented** – Use nouns, not verbs.
- **Stateless** – Each request contains all info.
- **Cacheable** – Use caching headers properly.
- **Secure** – Use auth, rate limiting, validation.
- **Versioned** – Handle backward compatibility.

REST API DESIGN EXAMPLE

RESOURCE	ENDPOINTS (EXAMPLES)		
Users	GET	/api/users	→ Get all users
	POST	/api/users	→ Create user
User	GET	/api/users/{id}	→ Get user by id
	PUT	/api/users/{id}	→ Update user
	DELETE	/api/users/{id}	→ Delete user
Products	GET	/api/products	→ Get all products
	POST	/api/products	→ Create product
Product	GET	/api/products/{id}	→ Get product by id
	PUT	/api/products/{id}	→ Update product
	DELETE	/api/products/{id}	→ Delete product

API DESIGN CHECKLIST

- ✓ Use meaningful resource names (nouns).
- ✓ Use proper HTTP methods.
- ✓ Use plural for collections (/users).
- ✓ Use status codes correctly.
- ✓ Return consistent response format.
- ✓ Support pagination, filtering & sorting.
- ✓ Version your APIs (/api/v1/).
- ✓ Provide clear error messages.

HTTP STATUS CODES (COMMON)

200	OK	– Success
201	Created	– Resource created
400	Bad Request	– Invalid input
401	Unauthorized	– Auth required
403	Forbidden	– No permission
404	Not Found	– Resource not found
429	Too Many Requests	– Rate limit
500	Internal Server Error	– Server issue

RESPONSE FORMAT (EXAMPLE)

Success Response

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Nikhil",
    "email": "nikhil@example.com"
  },
  "message": "User fetched successfully."
}
```

Error Response

```
{
  "success": false,
  "message": "User not found",
  "error_code": "USER_404"
}
```

PAGINATION, FILTERING & SORTING EXAMPLE

Pagination : /api/users?page=2&limit=10
→ Returns 10 users from page 2

Filtering : /api/users?role=admin&active=true
→ Returns active admin users

Sorting : /api/users?sort=createdAt&order=desc
→ Sort by created date (newest first)

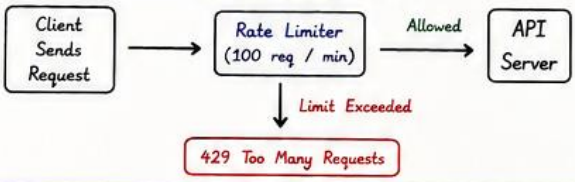
VERSIONING STRATEGIES

- **URI Versioning** : /api/v1/users
- **Header Versioning** : Accept-Version: v1
- **Query Versioning** : /api/users?version=1

→ URI Versioning is most common and simple.

RATE LIMITING EXAMPLE

Limit each user to 100 requests per minute.



AUTHENTICATION METHODS

API Key

Simple & easy. Good for server to server calls.

JWT (Token)

Stateless, secure. Used for user authentication.

OAuth 2.0

For third-party login (Google, GitHub, etc.)

KEY TAKEAWAYS

- ✓ Design APIs for developers.
- ✓ Follow REST principles and HTTP standards.
- ✓ Keep it simple, consistent and secure.
- ✓ Good API design = Happy developers!

★

INTERVIEW TIP

Interviewers may ask you to design an API. Always explain resources, endpoints, methods, status codes, auth, pagination and error handling.

☺

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0

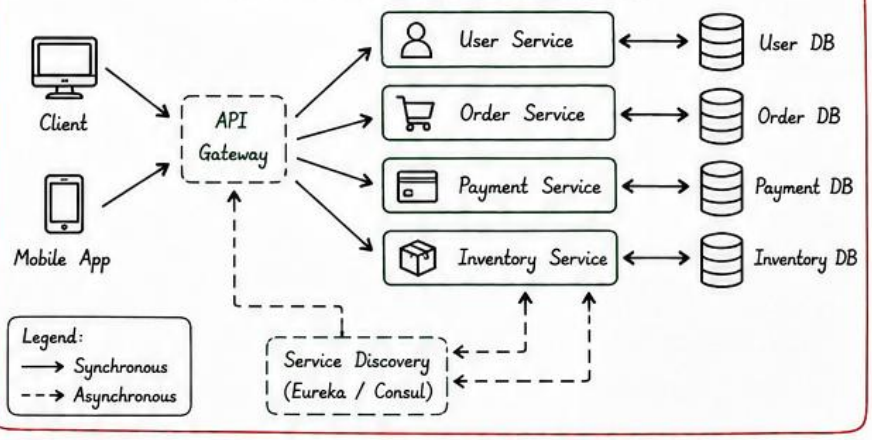
12. MICROSERVICES ARCHITECTURE BASICS

Microservices is an architectural style where an application is broken down into small, independent services that communicate over a network.
Each service is independently deployable, scalable and maintainable.

KEY CHARACTERISTICS

- Small & Independent Services
- Single Responsibility
- Independent Deployment
- Decentralized Data Management
- Resilience & Fault Isolation
- Technology Agnostic
- Scalability

TYPICAL MICROSERVICES ARCHITECTURE



MICROSERVICES PATTERNS

- API Gateway – Single entry point for clients
- Service Discovery – Find services dynamically
- Circuit Breaker – Prevent cascading failures
- Bulkhead – Isolate resources
- Saga Pattern – Manage distributed transactions
- Event Driven – Use events for communication
- CQRS – Separate read & write models

PROS

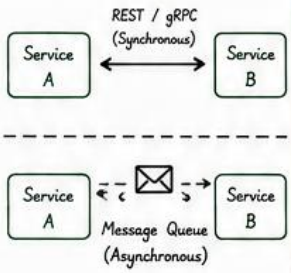
- ✓ Independent Deployment
- ✓ Technology Flexibility
- ✓ Scalability
- ✓ Fault Isolation
- ✓ Team Autonomy
- ✓ Better Maintainability

CONS

- ✗ Increased Complexity
- ✗ Network Latency
- ✗ Data Consistency Challenges
- ✗ Operational Overhead
- ✗ Testing is Harder
- ✗ Monitoring is Complex

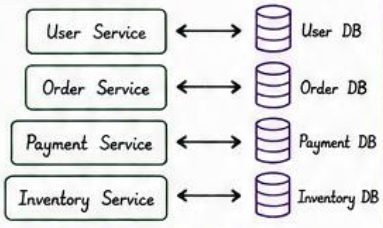
COMMUNICATION BETWEEN SERVICES

- Synchronous (Request/Response)
  - REST API, gRPC
- Asynchronous (Event Driven)
  - Message Queue, Pub/Sub
- Choose based on:
  - Use case
  - Performance
  - Reliability needs



DATA MANAGEMENT

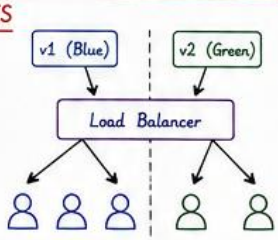
- Each service owns its own database.
- No shared database.
- Data is accessed only through the service.
- Use events or APIs for data sync.



Denormalization & eventual consistency are common.

DEPLOYMENT STRATEGIES

- Blue-Green Deployment
  - Zero downtime
- Canary Deployment
  - Rollout to small users first
- Rolling Deployment
  - Gradual rollout
- A/B Testing
  - Test new features safely



MONITORING & OBSERVABILITY

- Centralized Logging (ELK / Loki)
- Distributed Tracing (Jaeger / Zipkin)
- Metrics Collection (Prometheus / Grafana)
- Alerting & Dashboards



KEY TAKEAWAYS

- ✓ Microservices improve scalability and maintainability.
- ✓ But they bring complexity, so design carefully.
- ✓ Choose the right patterns and tools.
- ✓ Always monitor, test and document your services.

INTERVIEW TIP

Interviewers may ask you to compare Monolith vs Microservices, design a service, handle failures or choose the right communication pattern. Think in trade-offs and real-world scenarios.

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

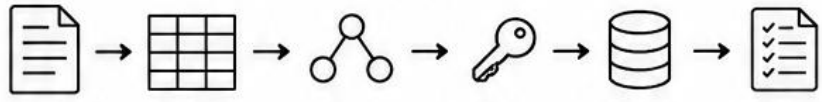
13. DATABASE DESIGN BASICS

Good database design ensures data integrity, performance and scalability.
It is the foundation of any backend system.

KEY PRINCIPLES

- Store only necessary data.
- Avoid redundancy.
- Ensure data integrity,
- Optimize for read/write performance.
- Design for future scale.

RELATIONAL DATABASE DESIGN FLOW

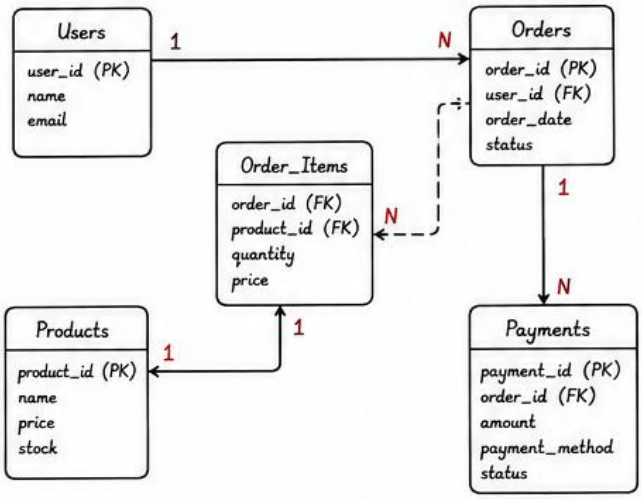


1. Requirements Analysis → 2. Identify Entities → 3. Define Relationships → 4. Choose Keys → 5. Normalize Tables → 6. Review & Optimize

NORMALIZATION (IN SIMPLE WORDS)

NORMAL FORM	DESCRIPTION
1NF	Each column has atomic (indivisible) values and unique rows.
2NF	In 1NF and no partial dependency. (Non-key attributes depend on the whole key)
3NF	In 2NF and no transitive dependency. (Non-key attributes depend only on keys)
BCNF	Stricter than 3NF. Every determinant is a candidate key.

EXAMPLE: E-COMMERCE DATABASE (HIGH LEVEL)



KEY TYPES

- Primary Key (PK): Uniquely identifies each row.
- Foreign Key (FK): Links to primary key of another table.
- Candidate Key: Minimal set of attributes that can uniquely identify rows.
- Composite Key: Combination of two or more attributes as key.

DATA TYPES (COMMON)

TYPE	EXAMPLE
INT	1, 42, -7
VARCHAR(n)	"Nikhil", "hello"
TEXT	Long text data
DATE	2025-07-17
DECIMAL(p, s)	999.99
BOOLEAN	true / false
JSON	{"key": "value"}

INDEXES

- Index helps in faster data retrieval.
- Like index in book.
- Too many indexes can slow down writes.

Types:

- Primary Index (on PK)
- Unique Index
- Composite Index
- Full Text Index

INDEX EXAMPLE

```
-- Create index on email column
CREATE INDEX idx_users_email
ON users(email);

-- Composite index
CREATE INDEX idx_order_user_date
ON orders(user_id, order_date);
```

ACID PROPERTIES

- Atomicity – All or nothing.
- Consistency – Always in valid state.
- Isolation – Transactions don't interfere.
- Durability – Once committed, stays forever.

DENORMALIZATION (WHEN & WHY)

- When reads are very frequent and joins are costly.
- Duplicate some data to improve performance.
- Use carefully. Can cause inconsistency.

COMMON MISTAKES

- ✗ Storing unnecessary data
- ✗ Not using proper data types
- ✗ Missing indexes on important columns
- ✗ Over-normalization
- ✗ Ignoring future scalability

BEST PRACTICES

- ✓ Understand requirements before designing.
- ✓ Use proper keys and relationships.
- ✓ Normalize up to the right level.
- ✓ Add indexes for frequently queried columns.
- ✓ Monitor performance and optimize queries.
- ✓ Backup data regularly.

INTERVIEW TIP

Always start with requirements, identify entities, relationships and cardinality.
Show your thought process. That's what interviewers care about!

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

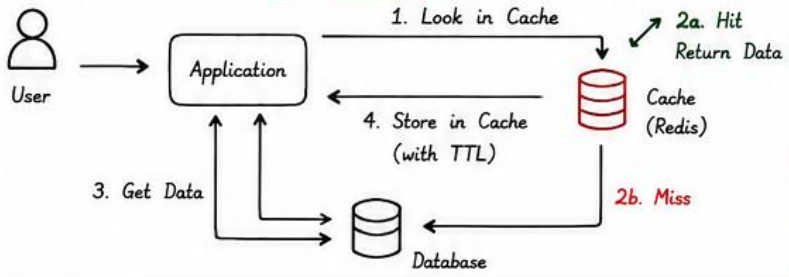
14. CACHING – CONCEPTS & STRATEGIES

Caching stores frequently accessed data in fast storage so that it can be served quickly, reducing latency and load on primary system.

WHY CACHE?

- Improves response time
- Reduces database load
- Saves bandwidth & compute
- Increases system throughput
- Improves user experience

CACHE FLOW



TYPES OF CACHE

TYPE	DESCRIPTION
In-Memory Cache	Stores data in RAM. Very fast. Example: Redis, Memcached
Distributed Cache	Shared across multiple servers. Example: Redis Cluster
Browser Cache	Stored in user's browser. Example: Local Storage, CDN Cache
CDN Cache	Stores static content closer to users. Example: Cloudflare, Akamai
Application Cache	Application level caching for frequently used data.

CACHE STRATEGIES

- **Cache-Aside (Lazy Loading)**
  App checks cache → if miss, fetch from DB and store.
- **Read-Through**
  Cache handles DB fetch on miss automatically.
- **Write-Through**
  Write to cache first, then DB.
- **Write-Back (Write-Behind)**
  Write to cache, update DB asynchronously.
- **Refresh-Ahead**
  Refresh cache before it expires.

EVICTION POLICIES

- **LRU (Least Recently Used)**
  Remove least recently used items.
- **LFU (Least Frequently Used)**
  Remove least frequently used items.
- **FIFO (First In First Out)**
  Remove the oldest items.
- **TTL (Time To Live)**
  Expire items after specified time.
- **Random**
  Remove random items.

CACHE KEY BEST PRACTICES

- ✓ Use a consistent and clear naming convention.
- ✓ Include user or tenant id if data is user specific.
- ✓ Keep keys short but meaningful.
- ✓ Use prefixes to avoid collisions.
- ✓ Example: `user:123:profile` or `product:456:details`

TTL BEST PRACTICES

- ✓ Set appropriate TTL based on data volatility.
- ✓ Short TTL for frequently changing data.
- ✓ Longer TTL for rarely changing data.
- ✓ Use sliding expiration for active sessions.
- ✓ Avoid very long TTL for critical data.

WHEN TO USE CACHE?

- Read-heavy workload
- Expensive DB queries
- Frequently accessed data
- Data that can tolerate slight staleness
- Rate limiting & session management

WHEN NOT TO USE CACHE?

- ✗ Write-heavy workload
- ✗ Highly dynamic and real-time data
- ✗ Data consistency is more important than speed
- ✗ Very large data that doesn't fit in memory
- ✗ When caching adds more complexity than benefit

CACHE INVALIDATION

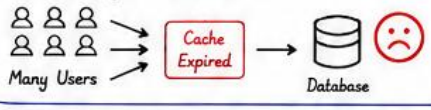
- Delete/expire cache on data update.
- Use versioning in key (e.g., `user:123:v2`).
- Event based invalidation (e.g., via message queue).
- Clear related caches, not everything.

COMMON PITFALLS

- ✗ Caching too much data
- ✗ Not handling cache expiration
- ✗ Cache stampede (many requests on miss)
- ✗ Storing sensitive data in cache
- ✗ Not monitoring cache hit/miss ratio

CACHE STAMPEDE (THUNDERING HERD) & SOLUTION

Problem:
When a hot key expires, many requests hit DB simultaneously and overload it.



Solutions:

- ✓ Add random TTL (jitter).
- ✓ Use mutex lock / distributed lock.
- ✓ Serve stale data while refreshing in background.
- ✓ Pre-warm cache (refresh-ahead).

METRICS TO MONITOR

- Cache Hit Ratio
- Cache Miss Ratio
- Evictions per second
- Memory Usage
- Average Latency
- Requests per second

INTERVIEW TIP Always mention trade-offs: consistency vs performance, memory vs latency. Think about data freshness, scalability and failure scenarios.

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0

15. MESSAGE QUEUES – CONCEPTS & BEST PRACTICES

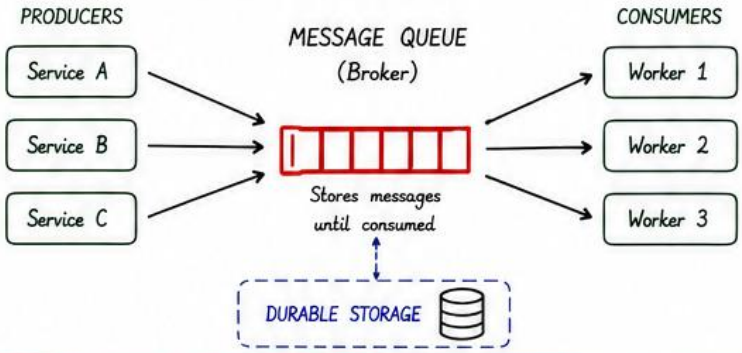
Message Queues enable asynchronous communication between services.
They decouple producers from consumers and make systems scalable & resilient.

WHY MESSAGE QUEUES?

- Asynchronous processing
- Decouples services
- Handles traffic spikes
- Improves reliability & availability
- Retry on failures
- Better resource utilization



HIGH LEVEL ARCHITECTURE



COMMON USE CASES

- Send email / notifications
- Order processing
- Payment processing
- Log aggregation
- File processing / transcoding
- Data sync between services



POPULAR MESSAGE QUEUES

- **RabbitMQ** – Widely used, supports many protocols.
- **Apache Kafka** – High throughput, persistent, distributed.
- **Amazon SQS** – Fully managed, integrates with AWS.
- **Redis Streams** – Lightweight, fast, in-memory.

MESSAGE QUEUE FLOW

1. Producer sends message to queue.
2. Queue stores message persistently.
3. Consumer reads & processes message.
4. On success, message is acknowledged and removed.
5. On failure, message can be retried or moved to DLQ.



MESSAGE DELIVERY SEMANTICS

1 At Most Once	Message may be lost but never delivered twice.	Use Case: Metrics, Logs
2 At Least Once	Message is delivered at least once, duplicates possible.	Use Case: Email, Orders
3 Exactly Once	Message is delivered once and only once.	Use Case: Payments, Financial txns

QUEUE TYPES

	Simple Queue	Messages are processed in FIFO order by single consumer.
	Work Queue (Competing Consumers)	Multiple consumers compete to process messages.
	Publish / Subscribe (Pub/Sub)	Messages are published to a topic and delivered to all subscribers.
↑↓	Priority Queue	Messages are processed based on priority.

BEST PRACTICES

- ✓ Keep messages small (avoid large payloads).
- ✓ Use meaningful message IDs for deduplication.
- ✓ Make consumers idempotent.
- ✓ Set appropriate visibility timeout / ack timeout.
- ✓ Use retries with exponential backoff.
- ✓ Monitor queue depth, lag & latency.
- ✓ Use DLQ (Dead Letter Queue) for failed messages.
- ✓ Encrypt sensitive data in messages.
- ✓ Define clear message schemas (JSON / Avro / Proto).



ERROR HANDLING & DLQ

- If consumer fails to process message multiple times.
- Move message to DLQ.
- Alert the team.
- Analyze and fix the issue.
- Reprocess from DLQ after fix.



RETRY STRATEGY

- **Immediate Retry** – For transient errors.
- **Exponential Backoff** – Wait and retry.
- **Max Retry Limit** – After limit, move to DLQ.
- **Jitter** – Add random delay to avoid thundering herd.



COMMON PITFALLS

- ✗ Messages too large – increases latency & memory usage.
- ✗ No idempotency – causes duplicate processing.
- ✗ Ignoring ack / visibility timeout – leads to message redelivery.
- ✗ No DLQ – makes debugging difficult.
- ✗ Single consumer bottleneck – limits throughput.
- ✗ Not monitoring queue metrics.



KEY METRICS TO MONITOR

- Queue Depth (Number of messages)
- Consumer Lag (Behind by how many)
- Message Age (Oldest message age)
- Throughput (Msgs produced / consumed)
- Error Rate / Retry Rate





INTERVIEW TIP Think about trade-offs: consistency vs availability, latency vs throughput, cost vs reliability. Always relate to real-world scale (millions of msgs/sec).



BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0 ★

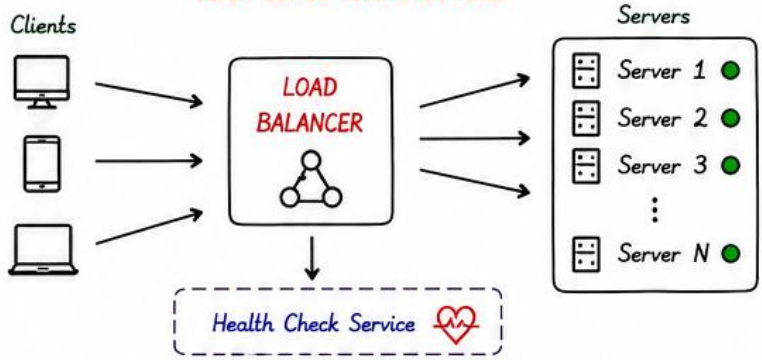
16. LOAD BALANCING – CONCEPTS & STRATEGIES

Load Balancing distributes incoming traffic across multiple servers to improve performance, availability and reliability.

WHY LOAD BALANCING?

- Distributes traffic evenly
- Avoids server overload
- Improves availability & reliability
- Enables horizontal scaling
- Supports zero downtime deployments

HIGH LEVEL ARCHITECTURE



TYPES OF LOAD BALANCERS

TYPE	DESCRIPTION
Hardware Load Balancer	Dedicated physical device. High performance, expensive.
Software Load Balancer	Runs on regular servers. Cost effective, flexible.
Cloud Load Balancer	Managed by cloud providers. Auto scaling, highly available.

LOAD BALANCING ALGORITHMS

- Round Robin – Requests in sequence.
- Weighted Round Robin – Based on server capacity.
- Least Connections – Requests to server with least active connections.
- IP Hash – Same IP goes to same server.
- Least Response Time – Based on server response time.
- Random – Randomly selects a server.
- Consistent Hashing – Used in caching systems & distributed setups.

ALGORITHM COMPARISON

ALGORITHM	BEST FOR
Round Robin	Uniform server capacity
Weighted RR	Servers with different capacities
Least Connections	Long lived connections
IP Hash	Session persistence
Least Response Time	Dynamic environments
Consistent Hashing	Caching / Sharding

LAYER OF LOAD BALANCING

- L4 (Transport Layer) – Uses IP & Port. Fast, no content awareness.
- L7 (Application Layer) – Uses HTTP/HTTPS info (URL, headers, cookies). More intelligent.

L4 – TCP/UDP Load Balancing
L7 – HTTP/HTTPS Load Balancing

HEALTH CHECKS

- Periodically checks server health.
- If server unhealthy → stop sending traffic.
- Types: HTTP, TCP, Ping, Custom script.
- Helps maintain high availability.

SESSION MANAGEMENT

- Without Sticky Session – Any server can handle next request.
- With Sticky Session – Same client always goes to same server.
- Methods: IP Hash, Cookies, Session DB, JWT.

SSL TERMINATION

- LB handles SSL/TLS encryption & decryption.
- Offloads SSL from backend servers.
- Improves performance.

FAILOVER STRATEGY

- Active-Passive – One active LB, other standby.
- Active-Active – Multiple LBs handling traffic.
- DNS Failover – Failover at DNS level.

STICKY SESSION METHODS

1. Source IP Hash
2. Cookie Based
3. Session ID in URL
4. External Session Store (Redis)

REAL WORLD TOOLS / SERVICES

- Nginx – Open source LB
- HAProxy – High performance LB
- AWS ALB / NLB
- GCP Load Balancer
- Azure Load Balancer

BEST PRACTICES

- ✓ Always use health checks.
- ✓ Choose right algorithm for your use case.
- ✓ Scale horizontally with auto scaling groups.
- ✓ Monitor metrics (CPU, connections, latency).
- ✓ Enable SSL termination at LB.
- ✓ Use sticky sessions only when required.
- ✓ Keep LBs across multiple AZs.

COMMON PITFALLS

- ✗ No health checks → sending traffic to down servers.
- ✗ Wrong algorithm choice → uneven load.
- ✗ No monitoring → issues go unnoticed.
- ✗ Single point of failure for LB.
- ✗ Not planning for SSL offloading.

KEY METRICS TO MONITOR

- Requests per second (RPS)
- Avg Response Time
- Healthy vs Unhealthy Servers
- Error Rate (4xx/5xx)
- Active Connections
- CPU & Memory Usage

INTERVIEW TIP

Always mention trade-offs while choosing strategies.
Talk about scalability, high availability and failure scenarios.
Share real world examples if you have used any LB (Nginx, ALB etc.).

BACKEND + INTERVIEW QUESTIONS

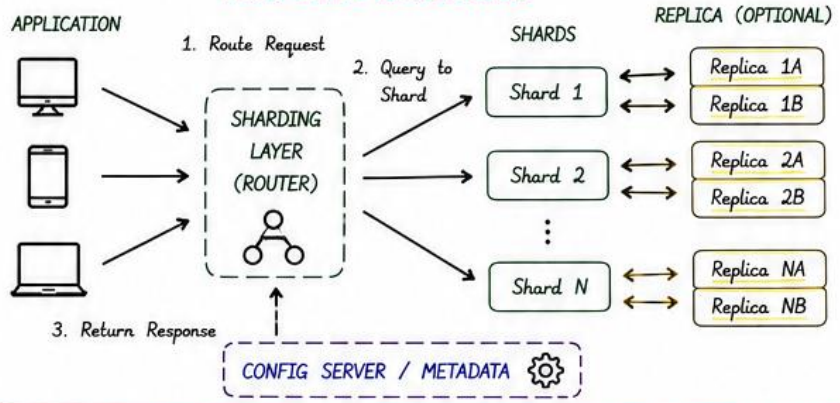
17. DATABASE SHARDING – CONCEPTS & STRATEGIES

Sharding splits a large database into smaller pieces called shards.
It improves scalability, performance and helps handle massive data.

WHY SHARDING?

- Handles growing data
- Distributes read/write load
- Improves performance
- Avoids single DB bottleneck
- Increases availability

HIGH LEVEL ARCHITECTURE



SHARDING STRATEGIES

STRATEGY	DESCRIPTION
1. Range Based	Data is split based on a range of values. (e.g., user_id 1-1000 → Shard 1)
2. Hash Based	Hash of shard key determines the shard. (e.g., user_id % N)
3. Directory Based	Uses a lookup table to map shard key to shard id.
4. Geo Based	Data is sharded based on geographic location.
5. Hybrid	Combination of two or more strategies.

SHARDING KEY SELECTION

- ✓ High cardinality (many unique values)
- ✓ Evenly distributed across data
- ✓ Frequently used in queries
- ✓ Immutable (should not change)
- ✓ Avoid monotonically increasing keys (if possible)

EXAMPLE – RANGE BASED SHARDING

USER_ID RANGE	SHARD
1 – 1,000	Shard 1
1,001 – 2,000	Shard 2
2,001 – 3,000	Shard 3
...	...
N-1,000 – N	Shard N

Note: Easy to understand but hard to handle hot spots.

EXAMPLE – HASH BASED SHARDING

Shard = Hash(user_id) % N

If N = 4

USER_ID	HASH(USER_ID)	SHARD
101	287612	0
102	981233	1
103	554331	3
104	776432	0
...	...	

SHARDING WORKFLOW

1. Application sends query with shard key.
2. Sharding layer calculates target shard.
3. Query is forwarded to that shard.
4. Shard processes query and returns result.
5. Result is sent back to application.

CHALLENGES

- ✗ Complex to implement and maintain
- ✗ Cross-shard joins are expensive
- ✗ Rebalancing shards is difficult
- ✗ Hotspot keys can cause imbalance
- ✗ Distributed transactions are harder

REBALANCING / RESHARDING

- When data grows, shards need to be split or moved.
- Steps:
  1. Add new shard(s).
  2. Move data from old shard(s) to new shard(s).
  3. Update routing / metadata.
  4. Handle traffic during migration.

DATA CONSISTENCY

- Strong Consistency – Slower, requires coordination.
- Eventual Consistency – Faster, data may be slightly stale.
- Use replication + quorum for better consistency.

SHARDING vs REPLICATION

SHARDING	REPLICATION
Horizontal Scale (Scale Out)	High Availability (Failover)
Splits Data	Copies Data
Improves Write Scalability	Improves Read Availability
Complex to Manage (Data Distribution)	Easy to Manage
Used for Performance	Used for Reliability

BEST PRACTICES

- ✓ Choose right shard key.
- ✓ Keep shard size balanced.
- ✓ Monitor shard metrics regularly.
- ✓ Plan for rebalancing.
- ✓ Use caching to reduce cross-shard calls.
- ✓ Test failure scenarios.

COMMON PITFALLS

- ✗ Poor shard key → Data imbalance.
- ✗ Too many small shards.
- ✗ Ignoring cross-shard queries.
- ✗ Not planning for re-sharding.
- ✗ No monitoring or alerts.

KEY METRICS TO MONITOR

- Data size per shard
- QPS per shard
- CPU / Memory usage
- Replication lag
- Error rate
- Hot keys / Unbalanced shards

INTERVIEW TIP

Explain trade-offs, shard key selection process & rebalancing approach.
Use real-world example (e.g., Twitter, Instagram, E-commerce).

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0

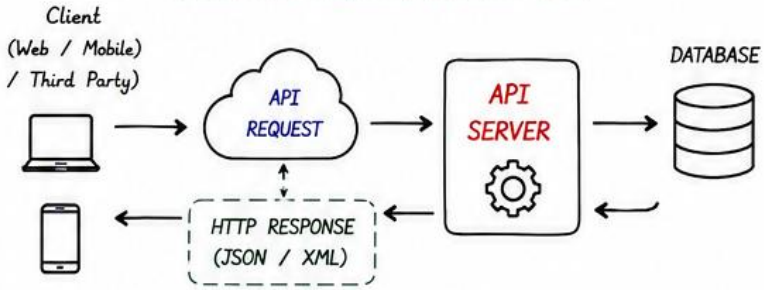
18. API DESIGN – BEST PRACTICES & PRINCIPLES

Good API design makes systems easy to use, scalable, secure and maintainable.
It is the contract between client and server.

WHY GOOD API DESIGN?

- Easy to understand & use
- Consistent experience
- Scalable & maintainable
- Reduces errors & ambiguity
- Speeds up development

TYPICAL API REQUEST/RESPONSE FLOW



API PRINCIPLES (REST)

PRINCIPLE	DESCRIPTION
Client–Server	Separation of concerns
Stateless	Each request from client contains all info needed
Cacheable	Responses should be cacheable where possible
Uniform Interface	Consistent, predictable interface
Layered System	Architecture can be composed of layers
Code on Demand (Optional)	Server can extend functionality (e.g., scripts)

BEST PRACTICES

- ✓ Use nouns for resources, not verbs.
  /users (Good) /getUsers (Bad)
- ✓ Use HTTP methods correctly.
- ✓ Keep URLs short, clean & consistent.
- ✓ Use plural for resources.
- ✓ Use pagination for large data.
- ✓ Implement filtering, sorting, searching.
- ✓ Version your APIs.
- ✓ Return meaningful status codes.
- ✓ Secure your APIs (Auth, Rate Limit, etc).
- ✓ Provide clear documentation.

HTTP METHODS

METHOD	PURPOSE
GET	Fetch resource
POST	Create resource
PUT	Update full resource
PATCH	Update partial resource
DELETE	Delete resource

STATUS CODES (COMMON)

2xx	Success	(200 OK, 201 Created)
3xx	Redirection	(301, 304)
4xx	Client Error	(400 Bad Request, 401 Unauthorized, 404 Not Found)
5xx	Server Error	(500 Internal Error, 503 Service Unavailable)

API ENDPOINT EXAMPLES

METHOD	ENDPOINT	DESCRIPTION
GET	/api/v1/users	Get all users
POST	/api/v1/users	Create a new user
GET	/api/v1/users/{id}	Get user by id
PUT	/api/v1/users/{id}	Update full user
PATCH	/api/v1/users/{id}	Update partial user
DELETE	/api/v1/users/{id}	Delete user

API RESPONSE FORMAT (JSON EXAMPLE)

Success Response (200 OK)

```
{
  "success": true,
  "data": {
    "id": 101,
    "name": "Nikhil Solanki",
    "email": "nikhil@example.com"
  },
  "message": "User fetched successfully"
}
```

Error Response (404 Not Found)

```
{
  "success": false,
  "error": "User not found",
  "code": 404
}
```

API VERSIONING STRATEGIES

- URI Versioning (Most common)
  /api/v1/users
- Header Versioning
  Accept: application/vnd.myapi.v1+json
- Query Parameter Versioning
  /api/users?version=1
- Don't break old clients.

FILTERING, SORTING & PAGINATION

- Filtering: GET /api/v1/users?role=admin
- Sorting: GET /api/v1/users?sort=name:asc
- Pagination: GET /api/v1/users?page=2&limit=10
  (page=2, limit=10)

SECURITY BEST PRACTICES

- Use HTTPS always.
- Authentication: JWT / OAuth2 / API Keys
- Authorization: Role based access.
- Rate limiting to prevent abuse.
- Validate & sanitize all inputs.
- Do not expose sensitive info.

COMMON MISTAKES

- ✗ Using verbs in URLs.
- ✗ Not following consistent naming.
- ✗ Returning wrong status codes.
- ✗ No pagination on large data.
- ✗ No versioning (breaks clients).
- ✗ Exposing internal server errors.

API DESIGN CHECKLIST

- ✓ Resource based URLs
- ✓ Correct HTTP methods used
- ✓ Proper status codes
- ✓ Request validation
- ✓ Response in standard format (JSON)
- ✓ Pagination, filtering, sorting supported
- ✓ Authentication & Authorization
- ✓ Rate limiting enabled
- ✓ API docs available (Swagger / OpenAPI)

API DOCUMENTATION (OPEN API / SWAGGER)

- Use OpenAPI (Swagger) to define & document APIs.
- Includes:
  - Endpoints
  - Request / Response schemas
  - Auth details
  - Example inputs / outputs
- Benefits: Auto docs, testing, client SDK generation.

TOOLS YOU SHOULD KNOW

- Postman – Testing APIs
- Swagger / OpenAPI – Documentation
- Insomnia – API client
- curl – Command line testing
- Rate limit tools – e.g., Redis, Nginx
- API Gateway – e.g., Kong, Nginx, AWS API Gateway

INTERVIEW TIP Always ask clarifying questions. Think about: inputs, outputs, errors, edge cases, security, scalability & performance.

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0

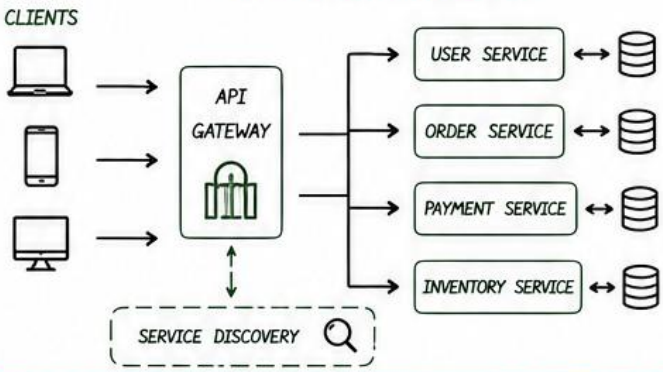
19. MICROSERVICES - CONCEPTS & BEST PRACTICES

Microservices break an application into small, independent services that communicate over a network. Each service is focused, loosely coupled and can be developed, deployed and scaled independently.

WHY MICROSERVICES?

- Independent development & deployment
- Technology freedom
- Scalability - scale what you need
- Fault isolation - one failure doesn't bring down everything
- Better team autonomy
- Faster time to market

HIGH LEVEL ARCHITECTURE



MICROSERVICES vs MONOLITH

ASPECT	MONOLITH	MICROSERVICES
Architecture	Single deployable unit	Small, independent services
Deployment	Deploy whole app	Deploy independently
Scaling	Scale entire app	Scale individual services
Technology	Single tech stack	Multiple tech stacks
Fault Isolation	Low - one failure affects all	High - failures are isolated
Team Structure	One big team	Small autonomous teams

MICROSERVICES PATTERNS

- API Gateway - Single entry point for clients.
- Service Discovery - Services find each other dynamically.
- Circuit Breaker - Prevents cascading failures.
- Bulkhead - Isolates resources to prevent failure spread.
- Saga Pattern - Manage distributed transactions.
- Event Driven - Use events for loose coupling.

COMMUNICATION PATTERNS

Synchronous (Request/Response)

- REST API (HTTP/HTTPS)
- gRPC
- GraphQL

Asynchronous (Event Driven)

- Message Queues (RabbitMQ, Kafka)
- Event Streaming (Kafka)
- Pub/Sub (SNS, NATS)

DATA MANAGEMENT

- Database per Service (Recommended) - Each service owns its data.
- Shared Database (Not Recommended) - Increases coupling.
- Use eventual consistency instead of strong consistency.

TRANSACTION MANAGEMENT

- Use Saga Pattern for distributed transactions.
- Two types:
  1. Choreography (Event based)
  2. Orchestration (Central coordinator)
- Compensating transactions to undo failed operations.

DEPLOYMENT STRATEGIES

- Blue-Green Deployment
- Canary Deployment
- Rolling Deployment
- Feature Flags
- Automated CI/CD pipelines

BEST PRACTICES

- ✓ Keep services small and focused.
- ✓ Follow single responsibility principle.
- ✓ Design APIs first, then implement.
- ✓ Use versioning for APIs.
- ✓ Implement proper logging & monitoring.
- ✓ Use health checks and graceful shutdown.
- ✓ Secure inter-service communication.

COMMON CHALLENGES

- ✗ Complexity in development & testing
- ✗ Distributed transactions management
- ✗ Network latency & failures
- ✗ Data consistency issues
- ✗ Service discovery & configuration
- ✗ Monitoring & debugging
- ✗ Higher operational overhead

TOOLS & TECHNOLOGIES

- API Gateway: Kong, Nginx, Traefik
- Service Discovery: Eureka, Consul, etcd
- Config Management: Spring Cloud Config
- Messaging: Kafka, RabbitMQ, NATS
- Monitoring: Prometheus, Grafana
- Logging: ELK Stack, Loki
- Containerization: Docker, Kubernetes

WHEN TO USE MICROSERVICES?

- ✓ Large and complex applications
- ✓ Multiple teams working independently
- ✓ Need independent scaling of components
- ✓ Different technology requirements
- ✓ High availability and fault isolation required

WHEN NOT TO USE?

- ✗ Small or simple applications
- ✗ Single team with limited resources
- ✗ High performance is not a critical need
- ✗ Need strong consistency across modules
- ✗ When operational complexity is a concern

KEY METRICS TO MONITOR

- Service Response Time (Latency)
- Error Rate (4xx/5xx)
- Throughput (Requests/sec)
- CPU, Memory Usage
- Network I/O
- Business Metrics

INTERVIEW TIP - While answering, focus on trade-offs, real-world examples, challenges and how you would solve them.

BACKEND + INTERVIEW QUESTIONS

Backend Handbook v1.0

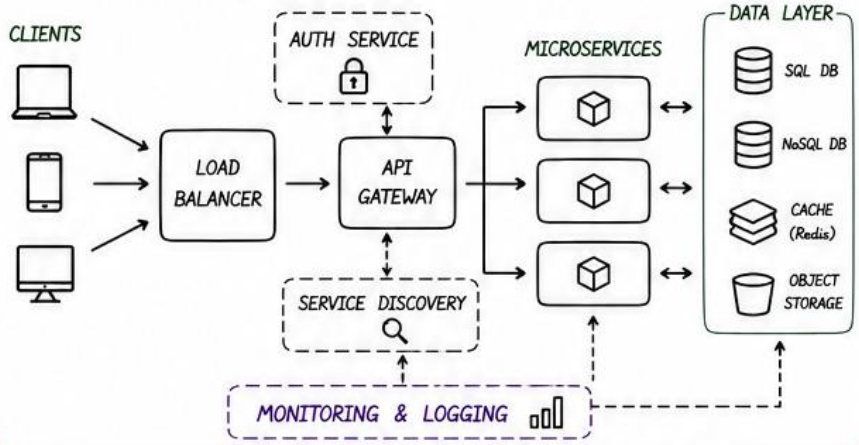
20. SYSTEM DESIGN - APPROACH & KEY TAKEAWAYS

System design is about making the right trade-offs to build systems that are scalable, reliable, available and maintainable.

SYSTEM DESIGN APPROACH

1. Clarify Requirements
   Ask questions, understand scope.
2. Define Goals & Constraints
   Functional, Non-functional, SLAs.
3. High Level Design
   Components, data flow, architecture.
4. Deep Dive Design
   Databases, APIs, algorithms, Protos.
5. Scale & Optimize
   Bottlenecks, scaling, caching, CDNs.
6. Wrap Up
   Trade-offs, alternatives, monitoring.

TYPICAL HIGH LEVEL ARCHITECTURE



NON-FUNCTIONAL REQUIREMENTS

- ✓ Scalability - Handle growth
- ✓ Availability - Minimize downtime
- ✓ Reliability - Consistent & correct
- ✓ Performance - Low latency, high throughput
- ✓ Security - Protect data & access
- ✓ Maintainability - Easy to change
- ✓ Cost - Optimize infrastructure cost

SCALING STRATEGIES

- Vertical Scaling - Add more power to a single server.
- Horizontal Scaling - Add more servers behind LB.
- Auto Scaling - Scale based on load.
- Database Sharding - Split data across multiple DBs.
- Caching - Reduce load on DB.
- CDN - Serve static content globally.

COMMON PATTERNS

- Cache Aside - App handles cache.
- Read Through - Read from cache first.
- Write Through - Write to cache & DB.
- Circuit Breaker - Prevent cascade failures.
- Retry with Backoff - Handle temporary failures.
- Idempotency - Same request, same result.

DATABASE DESIGN BEST PRACTICES

- Choose right database for the use case.
- Normalize when needed, denormalize for performance.
- Use proper indexing.
- Avoid N+1 queries.
- Connection pooling.
- Backup & replication.

API DESIGN BEST PRACTICES

- Use nouns for resources (not verbs).
- Use proper HTTP methods.
- Keep URLs clean & consistent.
- Version your APIs.
- Use proper status codes.
- Implement pagination, filtering, sorting.
- Rate limiting & authentication.

SECURITY BEST PRACTICES

- Use HTTPS always.
- Validate & sanitize all inputs.
- Use JWT / OAuth2 for auth.
- Authorize based on roles.
- Encrypt sensitive data.
- Keep secrets safe (env vars, vaults).

COMMON BOTTLENECKS

- ✗ Unindexed database queries
- ✗ Large payloads / responses
- ✗ Lack of caching
- ✗ Synchronous blocking calls
- ✗ Inefficient algorithms
- ✗ Poor network usage

TRADE-OFFS TO CONSIDER

TRADE-OFF	WHAT IT MEANS
Consistency vs Availability	CA or CP (CAP Theorem)
Latency vs Throughput	Optimize based on priority
Cost vs Performance	More power costs more
Simplicity vs Scalability	Simple now vs scale later
Real-time vs Reliability	Real-time systems are harder

MONITORING & OBSERVABILITY

- Monitoring - System health.
- Logging - What happened?
- Tracing - Request flow.
- Alerting - Get notified.
- Dashboards - Visual insights.

TOOLS YOU SHOULD KNOW

- Containers: Docker
- Orchestration: Kubernetes
- Caching: Redis
- Message Brokers: Kafka / RabbitMQ
- Monitoring: Prometheus, Grafana
- Logging: ELK / Loki / Graylog
- Cloud: AWS / GCP / Azure

INTERVIEW TIPS

- ✓ Ask clarifying questions.
- ✓ Think out loud.
- ✓ Start simple, then improve.
- ✓ Explain trade-offs.
- ✓ Draw clear diagrams.
- ✓ Communicate assumptions.
- ✓ Stay calm and structured.

KEY TAKEAWAYS

- ★ There is no perfect design - only trade-offs.
- ★ Focus on requirements & constraints.
- ★ Keep it simple, scalable and reliable.
- ★ Think for the user and the business.
- ★ Practice, build and learn continuously!

INTERVIEW TIP Great systems are built by great engineers who understand the problem, design the solution and adapt with time.